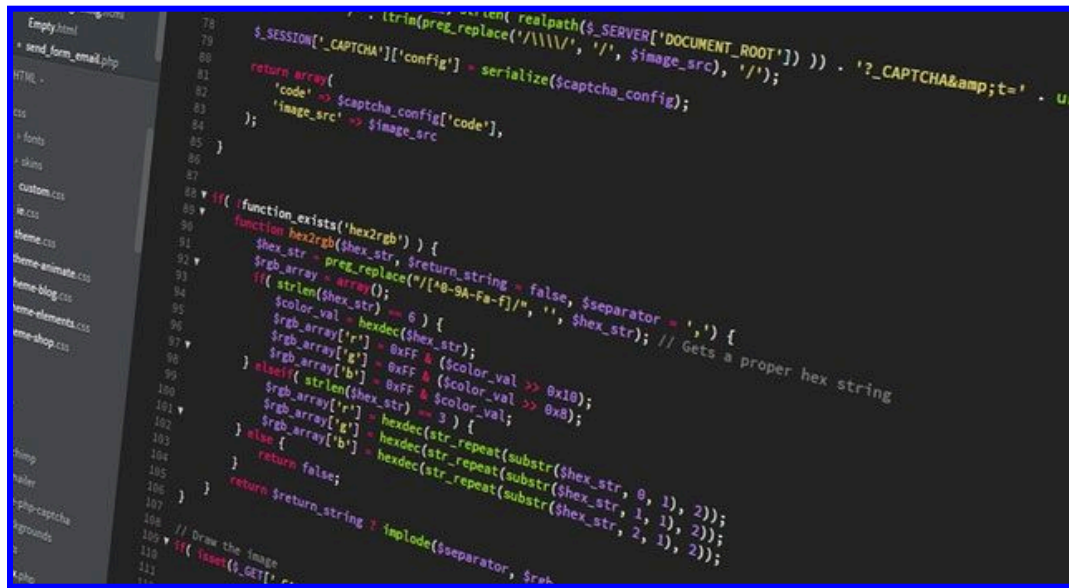


MEMORIA TÉCNICA

Gonzalo Prada Guzmán

GESTIÓN TIENDA INSTRUMENTOS

1ºDAM 26-05-2026



```
78 // ...
79 $SESSION['_CAPTCHA']['config'] = serialize($captcha_config);
80 return array(
81     'code' => $captcha_config['code'],
82     'image_src' => $image_src
83 );
84
85 }
86
87
88 // function_exists('hex2rgb')
89 function hex2rgb($hex_str, $return_string = false, $separator = ',') {
90     $hex_str = preg_replace("/[A0-9A-Fa-f]/", '', $hex_str); // Gets a proper hex string
91     $rgb_array = array();
92     if (strlen($hex_str) == 6) {
93         $color_val = hexdec($hex_str);
94         $rgb_array['r'] = 0xFF & ($color_val >> 0x10);
95         $rgb_array['g'] = 0xFF & ($color_val >> 0x8);
96         $rgb_array['b'] = 0xFF & ($color_val >> 0x0);
97     } elseif (strlen($hex_str) == 3) {
98         $rgb_array['r'] = hexdec(str_repeat(substr($hex_str, 0, 1), 2));
99         $rgb_array['g'] = hexdec(str_repeat(substr($hex_str, 1, 1), 2));
100         $rgb_array['b'] = hexdec(str_repeat(substr($hex_str, 2, 1), 2));
101     } else {
102         return false;
103     }
104     return $return_string ? implode($separator, $rgb_array) : $rgb_array;
105 }
106
107 // Draw the image
108 if (isset($_GET['...]) {
109     // ...
110 }
```

¿Qué es una memoria técnica?

Una memoria técnica de Java es, en pocas palabras, el plano y el manual de instrucciones definitivo de una aplicación desarrollada en Java. Es un documento formal donde se detalla cómo se construyó el software, por qué se tomaron ciertas decisiones de diseño y cómo funciona internamente.

Se enfoca en explicar la lógica del sistema para que cualquier otro desarrollador, o tu "yo" del futuro, pueda entenderlo, mantenerlo o escalar sin volverse loco.

Descripción del sistema

La aplicación es un **sistema de gestión para una tienda de instrumentos musicales**. Está pensada para el personal interno de la tienda, tiene funcionalidades para cada tipo de empleado, por lo que la mayoría de los usuarios internos de la tienda podrán usarlo, independientemente de su puesto, concretamente:

Para el jefe:

- Gestionar empleados (contratar, despedir, consultar rendimiento)
- Acceder protegido por contraseña
- Cambiar la contraseña del sistema

Para el personal en general:

- Consultar y gestionar el catálogo de instrumentos en venta (instrumentos de cuerda, instrumentos de viento o de percusión)
- Gestionar el inventario de artículos de uso personal del equipo
- Consultar proveedores
- Registrar trabajo realizado, y fichar salida y entrada
- Ver datos del trabajador

VENTAJAS DE SU USO:

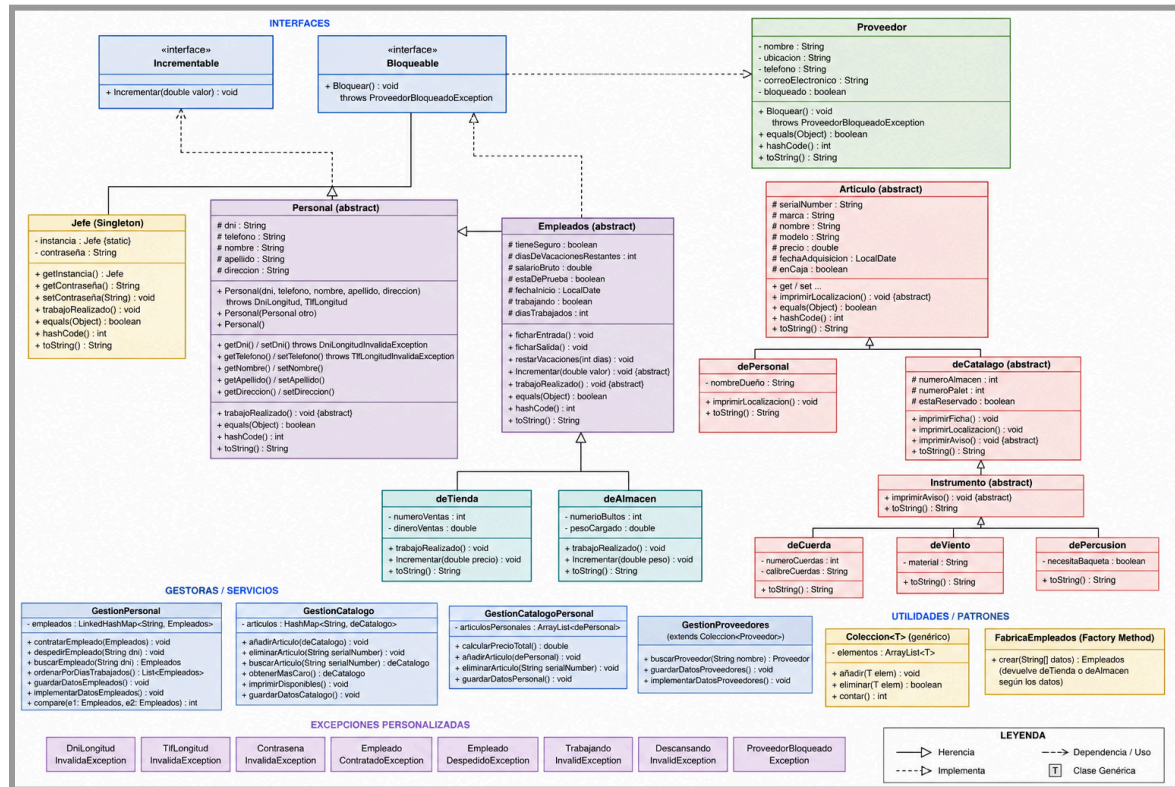
Es una herramienta de para una tienda de música que permite llevar el control de las tareas básicas que se realizan en negocios como este.

Agiliza trabajos y permite llevar un control de calidad en el negocio, debido a que es altamente personalizable para cada cliente.

Resuelve problemas en almacenes debido a que te avisa en que lugar exacto se encuentra dicho instrumento.

Análisis y diseño:

Para este proyecto he decidido realizar la siguiente jerarquía de clases:



Esquema realizado en DrawIO por **Gonzalo Prada Guzmán**, estilizado (Leyenda, colores y mejor posicionamiento) mediante IA Claude

JUSTIFICACIÓN DE DISEÑO

PERSONAL:

He tomado la decisión de realizar como clase raíz, la clase Personal, porque independientemente del tipo de personal, todos comparten mismos datos comunes:

Nombre, apellidos, dni, teléfono etc.....

Personal: es abstracta, debido a que no debería poder instanciarse, ya que más allá de sus datos comunes personales, no cuenta con datos relevantes sobre su puesto de trabajo en la empresa

Empleados: es una capa intermedia, entre personal y tipos de empleados, porque dentro del personal hay varios tipos de empleados, todos comparten ciertos datos comunes, por ello no puede instanciarse, porque faltarían datos relevantes para la empresa si se permite esto.

de Tienda/Almacén: acumulan los datos últimos necesarios por lo que no terminaría siendo abstracta, debido a que estas si deben instalarse. Estas clases son totalmente relevantes para el uso del programa (Añaden la interfaz "incremental" debido a que acumulan el peso y cantidad de ventas/bultos cargados).

Jefe: esta clase usa un **SINGLETON** debido a que solo se puede instanciar una vez (para eso funciona el singleton) debido a que es el único que tendrá el derecho a gestionar los empleados, por eso mismo tiene contraseña.

ARTÍCULO:

Usa como raíz la clase abstracta artículo debido a que usa datos comunes para todos los artículos, sean personal o de catálogo, como por ejemplo:

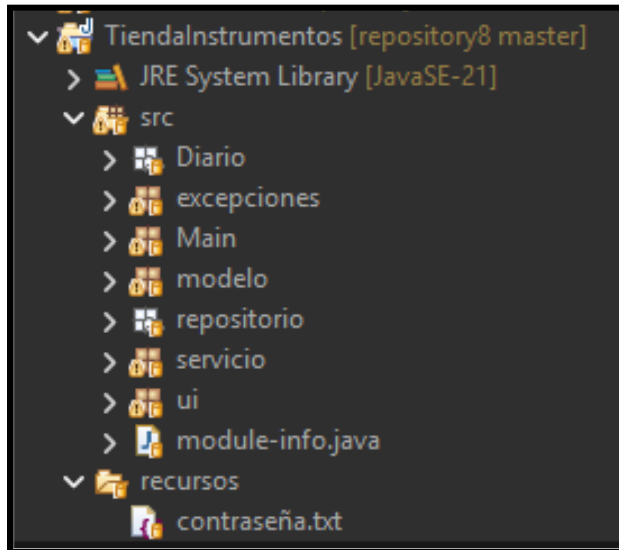
SN, nombre, precio, etc....

deCatalogo/Personal: he tenido que crear este contraste de clases, porque hay objetos que siguen naturalezas completamente diferentes, Catalogo es para la compra/venta de artículos, y Personal es para ver el inventario de los utensilios que usan el personal

Instrumento: es una capa intermedia entre Catálogo y los tipos de Instrumentos, debido a que no se podrá instanciar ningún instrumento sin saber el tipo que es, debido a que el comprador debe tener todos los datos a su disposición

de Cuerda/Viento/Percusión: última capa no abstracta (instanciable) que contiene toda la información necesaria para el comprador.

ARQUITECTURA DE PAQUETES



Esta arquitectura de paquete se justifica teniendo en cuenta lo siguiente:

- DIARIO:** Aquí va un txt con un modelo TO-DO con fechas y funciones de implementación
- EXCEPCIONES:** Aquí van las excepciones que vamos a utilizar a lo largo del código
- MAIN:** Aquí se encuentra la clase main, esto es debido a que no es interesante para el código que la main esté suelta
- MODELO:** aquí se encuentran las clases, sub clases e interfaces interesantes para la creación del código
- REPOSITORIO:** aquí se encuentran los txt, para el guardado o eliminación de datos dinámicos.
- SERVICIO:** aquí se encuentran las clases CRUD (gestoras) necesarias para el funcionamiento del código en la main

-UI: aquí se encuentran las clases que se ejecutan desde la main para que funcione el código exactamente.

MANUAL USUARIO:

Ahora procederemos a explicar a detalle cada una de las pantallas del programa y como funciona:

-Lo primero que veremos al iniciar el programa en la main (mediante el paquete main) sera la siguiente pantalla

```
[OK] Empleados cargados.  
[OK] Catalogo cargado.  
[OK] Inventario personal cargado.  
[OK] Proveedores cargados.  
  
TIENDA DE INSTRUMENTOS  
Bienvenido, Jefe  
  
1. Empleados  
2. Inventario  
0. Salir (guardar y cerrar)  
  
Selecciona una opcion:
```

Esta es la pantalla principal, como podremos comprobar en la parte superior comprobaremos que todo haya sido cargado correctamente, (en el caso de que la contraseña no cargue se mantendrá de defecto: **admin1234**.)

```
--- Acceso restringido: area de empleados ---  
Introduce la contraseña del jefe: admin1234
```

Si todo sale bien podremos empezar a ejecutar el programa, para ello elegiremos la opción que deseemos:

-1 Gestión de empleados: sirve realizar cambios a los empleados y visualizar datos sensibles, a este apartado sólo tendrán acceso los administradores, por lo que habrá que acceder con contraseña

-2 Inventario: estos datos son comunes y accesibles para todos los empleados, en el se encuentra la gestión de localización y datos de objetos, ya sean, instrumentos a la venta, u material de oficina inventariado.

-0 Salir: se guardarán todos los cambios que hayamos realizado **IMPORTANTE:** no se guardará ningún cambio si sales del programa sin acceder a Salir (0).

En el caso de que ingreses una opción no válida:

```
Selecciona una opcion: d  
[!] Opcion no valida. (1, 2 o 0).
```

GESTIÓN DE EMPLEADOS:

```
=====
MENU - GESTION DE EMPLEADOS
=====
1. Empleados de Tienda
2. Empleados de Almacen
3. Contratar nuevo empleado
4. Despedir empleado
5. Ver todos los empleados (ordenados)
6. Cambiar contraseña
7. Guardar datos de empleados
0. Volver al menu principal
=====
Selecciona una opcion:
```

- 1) **Empleados de tienda:** aquí accederemos al menú de empleados
- 2) **Empleados de almacén:** aquí accederemos al menú de almacén

- 3) **Contratar nuevo empleado:** aquí nos pedirán los datos necesarios para contratar un nuevo empleado, hay que tener en cuenta, que cierto datos como dni, o telefono no pueden repetirse, por lo que si añades un dato de este tipo que ya se encuentra en la lista de datos, saltará y te pedirá añadirlo otra vez.
- 4) **Despedir empleado:** accederemos a un menú que pide DNI del empleado para proceder con su borrado de datos.
- 5) **Ver todos los empleados:** aquí veremos todos los empleados ordenados alfabéticamente
- 6) **Cambiar contraseña:** nos pedirá añadir contraseña actual y añadir la nueva contraseña, la contraseña por defecto es "admin1234".
- 7) **Guardar datos de empleados:** aqui podremos hacer un guardado de los cambios realizados en el caso de que no queramos pasar por la opción 0 (salir y guardar)
- 8) **Volver al menú principal:** volveremos a la pantalla anterior

1-EMPLEADOS TIENDA

```
=====
MENU - EMPLEADOS DE TIENDA
=====
1. Ver todos los empleados de tienda
2. Registrar venta
3. Fichar entrada
4. Fichar salida
5. Ver trabajo realizado (por DNI)
0. Volver
=====
Selecciona una opcion:
```

- 1) **Ver todos los empleados de tienda:** imprimirá los empleados que son solo de tienda, en el caso de que no haya, se hará saber
- 2) **Registrar venta:** identificamos al usuario y pediremos el beneficio sacado, para almacenar estos datos

- 3) **Fichar entrada:** pediremos el DNI del usuario y se marcara como **trabajando**, si ya ha fichado la entrada pero no la salida, esto no funcionara
- 4) **Fichar salida:** pediremos el DNI del usuario y se marcara como **descansando**, tras esto se almacenará como día trabajado
- 5) **Ver trabajo realizado:** se pedirá el dni de dicho usuario e imprimimos cuantas ventas con cuánto beneficio ha generado, y cuantos dias ha trabajado, además de si esta en prácticas o no

2-EMPLEADOS ALMACÉN

```
=====
MENU - EMPLEADOS DE ALMACEN
=====
1. Ver todos los empleados de almacen
2. Registrar bulto (Incrementar)
3. Fichar entrada
4. Fichar salida
5. Ver trabajo realizado (por DNI)
0. Volver
=====
Selecciona una opcion:
```

- 6) **Ver todos los empleados de almacén:** imprimirá los empleados que son solo de almacén, en el caso de que no haya, se hará saber
- 7) **Registrar bulto:** identificamos al usuario y pediremos los kilos cargados, para almacenar estos datos
- 8) **Fichar entrada:** pediremos el DNI del usuario y se marcara como **trabajando**, si ya ha fichado la entrada pero no la salida, esto no funcionara
- 9) **Fichar salida:** pediremos el DNI del usuario y se marcara como **descansando**, tras esto se almacenará como día trabajado
- 10) **Ver trabajo realizado:** se pedirá el dni de dicho usuario e imprimimos cuantos bultos a cargado con cuántos kg, y cuántos días ha trabajado, además de si esta en prácticas o no.

GESTION INVENTARIO

```
=====
MENU - INVENTARIO
=====
1. Catalogo de instrumentos (Jefe)
2. Articulos del personal
0. Volver al menu principal
=====
Selecciona una opcion:
```

- 1) **Catalogo de instrumentos:** accederemos a ver el inventario de los instrumentos en venta
- 2) **Articulos del personal:** accederemos a un inventario de objetos personales de los empleados de dicha empresa

1-CATALOGO DE INSTRUMENTOS

```
=====
MENU - CATALOGO DE INSTRUMENTOS
=====
1. Ver todos los articulos
2. Ver articulos disponibles
3. Ver articulo mas caro
4. Contar disponibles
5. Buscar articulo (por S/N)
6. Añadir instrumento de Cuerda
7. Añadir instrumento de Viento
8. Añadir instrumento de Percusion
9. Reservar / Liberar articulo
10. Eliminar articulo
11. Guardar catalogo en fichero
0. Volver
```

- 1) **Ver todos los artículos:** imprimirá por pantalla los artículos a la venta dando todos sus datos (estén reservados o no).
- 2) **Ver artículos disponibles:** imprimirá por pantalla los artículos a la venta que estén disponibles (solo no reservados)
- 3) **Ver artículo más caro:** imprimirá el artículo más cara independientemente de si está reservado
- 4) **Contar disponibles:** para el ahorro de tiempo, se realizará un conteo de instrumentos disponibles por pantalla
- 5) **Buscar artículo:** se imprimirá una ficha técnica de datos del instrumento buscando por un s/n ingresado por consola
- 6) **Añadir instrumento de cuerda:** se pedirán los datos necesarios para añadir un instrumento y será marcado como "de Cuerda"
- 7) **Añadir instrumento de viento:** se pedirán los datos necesarios para añadir un instrumento y será marcado como "de Viento"
- 8) **Añadir instrumento de percusión:** se pedirán los datos necesarios para añadir un instrumento y será marcado como "de Percusión"
- 9) **Reservar/liberar artículo:** se ingresara S/N, en el caso de que dicho artículo esté disponible, dejará de estarlo, en el caso de que esté reservado, se liberara
- 10) **Eliminar artículo:** se ingresara un S/N, tras esto dicho artículo dejara de existir en el programa
- 11) **Guardar catalogo en el fichero:** en el caso de que quieras guardar sin cerrar el programa puedes hacerlo mediante esta función

2-ARTÍCULOS PERSONAI

```
=====
MENU - ARTICULOS DEL PERSONAL
=====
1. Ver todos los articulos
2. Buscar articulo (por S/N)
3. Añadir articulo de personal
4. Eliminar articulo
5. Calcular precio total del inventario personal
6. Guardar datos en fichero
0. Volver
=====
```

- 1) **Ver todos los artículos:** imprimirá por pantalla los artículos del personal dando todos sus datos.
- 2) **Buscar artículo:** se imprimirá datos del artículo buscando por un s/n ingresado por consola
- 3) **Añadir artículo personal:** se pedirán los datos necesarios para añadir un artículo de personal
- 4) **Eliminar artículos:** se pedirá el s/n del artículo, si existe se elimina
- 5) **Calcular el precio total del inventario personal:** se imprimirá por pantalla el total gastado en material ofimático del personal.
- 6) **Guardar datos en fichero:** en el caso de que quieras guardar sin cerrar el programa puedes hacerlo mediante esta función

EXPLICACIÓN TÉCNICA DE CONCEPTOS APLICADOS

Programación Orientada a Objetos y Encapsulamiento

El encapsulamiento es algo muy importante en todas las clases de nuestro modelo. Los atributos se declaran como privados o protegidos. Por ejemplo, en las clases `Personal` y `Empleados`, los datos como el DNI, el teléfono o el salario solo se pueden modificar mediante métodos especiales llamados `setters`. En estos `setters`, también se realizan validaciones para asegurarnos de que los datos sean correctos.

En el método `Personal.setDni()`, se comprueba que el DNI tenga 9 caracteres. En `Empleados.setSalarioBruto()`, se evita asignar salarios inferiores a 800. También se han implementado métodos como `equals()`, `hashCode()` y `toString()` en clases como `Personal`, `Articulo` y `Proveedor` para permitir comparaciones.

Herencia y Polimorfismo

La herencia se utiliza principalmente en dos jerarquías: **Personal** → **Empleados** → **deTienda/deAlmacen y Articulo** → **deCatalogo** → **Instrumento** → **deCuerda/deViento/dePercusion**. Las clases hijas pueden reutilizar atributos y métodos comunes definidos en las clases padre.

El polimorfismo se aplica en las gestoras, por ejemplo en `GestionPersonal`, donde todos los empleados se almacenan como `Empleados`, aunque realmente sean `deTienda` o `deAlmacen`. Cuando se llama a métodos como `trabajoRealizado()` o `Incrementar()`, Java ejecuta automáticamente la versión correspondiente al tipo real del objeto.

Clases Abstractas e Interfaces

Se han implementado clases abstractas en Personal, Empleados, Articulo, deCatalogo e Instrumento, ya que representan conceptos generales que no deben crearse directamente. Estas clases obligan a las subclases a implementar métodos concretos, como `trabajoRealizado()` o `imprimirAviso()`.

Además, se han desarrollado las interfaces Incrementable y Bloqueable. Incrementable se implementa en Empleados para obligar a cada tipo de empleado a definir su propia lógica de incremento. Bloqueable se usa en Proveedor para controlar el bloqueo de proveedores.

Colecciones y Genéricos

En las gestoras se han utilizado distintas colecciones según las necesidades de cada caso. GestionPersonal usa `LinkedHashMap<String, Empleados>` para buscar empleados rápidamente por DNI y mantener el orden. GestionCatalogo utiliza `HashMap<String, deCatalogo>` porque el orden no es importante y se prioriza el rendimiento.

GestionCatalogoPersonal usa `ArrayList<dePersonal>` debido a que los artículos personales se manejan como una lista secuencial. También se ha implementado la clase genérica `Coleccion<T>`, utilizada como base en GestionProveedores para reutilizar operaciones comunes sin duplicar código.

Gestión de Excepciones

Se han implementado excepciones personalizadas para controlar errores específicos del sistema. Por ejemplo, `DniLongitudInvalidaException` se lanza en `Personal.setDni()` y `TlfLongitudInvalidaException` en los setters de teléfono.

En `GestionPersonal` se utilizan excepciones como `EmpleadoContratadoException` o `EmpleadoDespedidoException` para controlar errores al gestionar empleados. Además, métodos como `ficharEntrada()` y `ficharSalida()` lanzan excepciones si el empleado ya estaba trabajando o descansando.

Todas estas excepciones se capturan en la capa ui mediante try-catch para evitar cierres inesperados del programa.

Persistencia en Ficheros

La persistencia se implementa mediante archivos de texto gestionados por cada gestora. `GestionPersonal` trabaja con `empleados.txt`, `GestionCatalogo` con `catalogo.txt`, `GestionCatalogoPersonal` con `personal.txt` y `GestionProveedores` con `proveedores.txt`.

Para leer y escribir datos se utilizan `BufferedReader` y `BufferedWriter`, permitiendo almacenar todos los objetos entre ejecuciones. Al iniciar la aplicación, métodos como `implementarDatosCatalogo()` reconstruyen automáticamente los objetos leyendo el tipo almacenado en cada línea del fichero.

Streams y Expresiones Lambda

La Stream API se ha implementado para realizar consultas y operaciones sobre colecciones de forma más eficiente y legible. En GestionPersonal, métodos como `filtrarEmpleadosTiendaImprimir()` utilizan `filter()` para mostrar solo empleados de tienda.

En GestionCatalogo, `count()` se utiliza para contar artículos disponibles y `max()` para obtener el artículo más caro. Finalmente, `GestionCatalogoPersonal.calcularPrecioTotal()` usa `map()` y `reduce()` para calcular el valor total de los artículos personales.

Patrones de Diseño

El patrón Singleton se ha implementado en la clase Jefe, donde el constructor es privado y la única forma de obtener la instancia es mediante `Jefe.getInstance()`. Esto garantiza que solo exista un administrador en toda la aplicación.

Por otro lado, el patrón Factory Method se aplica en `FabricaEmpleados`, encargada de crear objetos de `Tienda` o de `Almacen` según los datos leídos desde fichero. De esta forma, `GestionPersonal` no necesita conocer los detalles de creación de cada tipo de empleado.

DIFICULTADES ENCONTRADAS

Un proyecto de tal calibre, por lo general, tiene diversas dificultades, y este, no es una excepción;

He tenido varias dificultades técnicas, pero las más destacables han sido las siguientes:

SINGLETON:

-A la hora de crear la clase jefe, ha surgido un problema, y es que para que el diseño tenga sentido, no puede haber más de un jefe (por concluyente más de una contraseña).

-Esto hace complicado su creación, debido a que no requiere conocimiento para realizar un bloqueo de instanciamiento más que uno.

-Investigando conocí los singleton, una forma de constructor que permite que no se pueda instanciar más de una vez.

FUNCIONES MUY LARGAS A LA HORA DE GUARDAR/CARGAR DATOS

-Esto se dio sobretodo en las clases gestoras debido que al ser muchos datos lo que se deben cargar

-Esto se solucione mediante un array con los datos necesarios, así en vez de meter datos sueltos, metes el array con todo lo necesario para el constructor

CONCLUSIONES

DOCUMENTACIÓN:

He llegado a la conclusión que para proyectos de este tipo se debe realizar una documentación diaria, como por ejemplo un To-Do, sin el uso de este no hubiera conseguido implementar funciones o me hubieran costado mucho más, para siguientes proyectos voy a seguir implementando esta metodología.

APRENDIZAJE:

Con estos tipos de proyectos he llegado a la conclusión que se aprende mediante fallos, como por ejemplo el antes mencionado SINGLETON, que tuve que aprender a usarlo porque el modelo que realizaba lo requería. Por lo que para estos proyectos hay que tener mente abierta, e ir con la mentalidad de que se necesitarán implementar herramientas que no conocemos

REFERENCIAS

En el mismo código hay referencias a apartados de código que he necesitado ayuda implementar, para ello me he documentado mediante ciertas IAs (las zonas con ia están marcadas) e video de youtube, estos son dichos videos:

TODO CODE

<https://www.youtube.com/watch?v=UekxC1hvrk>

https://www.youtube.com/watch?v=OX_NRjRqoWM

https://www.youtube.com/watch?v=Z9Z_umK4bJc

MAKIGAS

<https://www.youtube.com/watch?v=0ZMaASgbSAg>

